

UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO



Gestão do Conhecimento em Sistemas de Software: Uma Ferramenta Baseada em Análise Estática de Código

Recife
2026

Marley Pinheiro Martins

Gestão do Conhecimento em Sistemas de Software: Uma Ferramenta Baseada em Análise Estática de Código

Relatório Técnico apresentado ao Curso de Bacharelado em Sistemas de Informação da Universidade Federal Rural de Pernambuco, como requisito parcial para obtenção do título de Bacha- rel em Sistemas de Informação.

Orientadora: Katarina Leal Chaves Lacerda
(DADM/UFRPE)

Resumo

A Engenharia de Software enfrenta desafios crescentes à medida que sistemas modernos ganham escala e complexidade. Equipes distribuídas trabalham sobre bases de código extensas, compostas por módulos interdependentes e decisões arquiteturais acumuladas ao longo do tempo. O domínio sobre esse sistema raramente está centralizado. Distribui-se entre desenvolvedores experientes, históricos de alterações e, frequentemente, na memória individual de quem o construiu. A alta rotatividade característica do setor agrava esse cenário, tornando a integração de novos colaboradores lenta e expondo as equipes à perda contínua de domínio técnico. Isso evidencia uma necessidade fundamental relacionada ao fato de que o conhecimento construído ao longo do desenvolvimento, desde decisões de arquitetura, padrões adotados, lógica dos módulos, seja preservado e transmitido de forma eficaz. Nessa perspectiva, é necessário garantir que esse domínio não fique restrito a indivíduos e se torne acessível ao coletivo. É nessa lacuna que a Gestão do Conhecimento oferece contribuições fundamentais. Ao tratar o conhecimento organizacional como um ativo estratégico a ser capturado, estruturado e compartilhado, esse campo revela que a perda de domínio técnico não é apenas um problema de Engenharia de Software, mas também organizacional. Diante disso, o objetivo deste trabalho é desenvolver uma ferramenta para apoiar a preservação e o compartilhamento do conhecimento em projetos de *software*, utilizando análise estática de código para extrair informações sobre a estrutura dos sistemas, facilitar a disseminação do conhecimento técnico e reduzir sua dependência de indivíduos específicos. Para isso, foi adotada uma abordagem de desenvolvimento de uma aplicação baseada em análise estática de código, utilizando a leitura da Árvore de Sintaxe Abstrata (AST) integrada a algoritmos de Inteligência Artificial para a sumarização automatizada de módulos de código e a geração dinâmica de grafos de dependência. Como contribuição esperada, o trabalho visa fornecer um mecanismo automatizado que reduz a curva de aprendizado de novos membros, documenta o sistema de forma contínua e mitiga a perda de conhecimento crítico diante da rotatividade de profissionais.

1. INTRODUÇÃO

A crescente complexidade dos sistemas de *software* contemporâneos transformou o desenvolvimento tecnológico em um esforço massivamente colaborativo e intrincado. Diferentemente de décadas passadas, o *software* moderno não é construído por indivíduos isolados, mas por equipes cada vez maiores e, frequentemente, distribuídas geograficamente (Pressman; Maxim, 2021). Essa escala traz consigo um desafio inerente: a dependência crítica do conhecimento técnico acumulado durante o ciclo de vida do desenvolvimento. Decisões arquiteturais, regras de negócio complexas e padrões de codificação específicos muitas vezes residem exclusivamente na mente dos desenvolvedores seniores, configurando um conhecimento tácito e não documentado.

Neste cenário, a alta rotatividade de profissionais, característica marcante do setor de Tecnologia da Informação, torna-se um risco operacional (Sommerville, 2019). Quando um desenvolvedor experiente deixa a organização, ele leva consigo não apenas suas habilidades, mas o domínio do contexto histórico e estrutural do projeto, porque determinada arquitetura foi escolhida em detrimento de alternativas, quais requisitos de negócio motivaram decisões que hoje parecem arbitrárias, quais são as partes frágeis do sistema que exigem atenção redobrada. A dificuldade de preservar esse conhecimento arquitetural e essas decisões técnicas gera consequências profundas para as organizações. Esse fenômeno remete à perda da memória organizacional, um risco alertado por Davenport e Prusak (1998), que ocorre de forma progressiva quando colaboradores deixam a empresa sem que seu conhecimento tácito tenha sido devidamente capturado, estruturado e preservado. As consequências práticas dessa amnésia corporativa são profundas e multidimensionais, tempo necessário para o treinamento e a integração (*onboarding*) de novos desenvolvedores aumenta consideravelmente. A manutenção do *software* torna-se mais lenta, propensa a erros e financeiramente custosa, uma vez que novos programadores precisam decifrar o sistema por tentativa e erro, e funcionalidades são reimplementadas por desconhecimento (Martin, 2009). Essa perda contínua de domínio técnico compromete a sustentabilidade e a evolução do *software* a longo prazo, configurando um problema organizacional e financeiro, além de técnico.

Dessa forma, o presente trabalho justifica-se pela necessidade de mitigar

essa volatilidade do conhecimento em equipes de engenharia de *software*, substituindo a dependência da memória individual por processos automatizados de extração e registro de informações. A relevância científica deste estudo reside na interseção entre a Engenharia de Software e a Gestão do Conhecimento, propondo formas inovadoras de documentação autônoma. Do ponto de vista prático, oferece uma solução tangível para que empresas de software protejam seu capital intelectual e otimizem seus recursos operacionais.

Portanto, o objetivo geral desta pesquisa é desenvolver uma ferramenta baseada em análise estática de código, capaz de extrair, estruturar e sumarizar automaticamente o conhecimento arquitetural de sistemas de *software*, promovendo a gestão do conhecimento técnico nas equipes de desenvolvimento. Para alcançar este propósito, definem-se os seguintes objetivos específicos:

1. Mapear as dependências e a estrutura dos sistemas de software através da leitura da Árvore de Sintaxe Abstrata (AST);
2. Aplicar modelos de Inteligência Artificial para a sumarização automatizada das regras de negócio e funcionalidades dos módulos de código analisados;
3. Disponibilizar o conhecimento extraído por meio de uma interface visual interativa (grafos de dependência e catálogos), facilitando a recuperação da informação pelos desenvolvedores.

2.REFERENCIAL TEÓRICO

2.1 Sistemas de Informação

Os Sistemas de Informação (SI) podem ser compreendidos como conjuntos integrados de componentes, sendo eles, pessoas, processos, tecnologia e dados que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisão e o controle nas organizações (Laudon; Laudon, 2014). Para aprofundar essa compreensão, o Quadro 1 apresenta diferentes perspectivas teóricas sobre o conceito de SI.

Quadro 1 - Perspectivas teóricas sobre Sistemas de Informação

Autor(es) / Ano	Conceito de Sistema de Informação
Laudon e Laudon (2014)	Conjunto de componentes inter-relacionados que coletam, processam, armazenam e distribuem informações para apoiar a tomada de decisões e o controle em uma organização.
O'Brien e Marakas (2013)	Uma combinação organizada de pessoas, hardware, software, redes de comunicação e recursos de dados que coleta, transforma e dissemina informações.
Stair e Reynolds (2020)	Um conjunto de elementos inter-relacionados que coletam dados processados, convertendo-os em informações precisas, fornecendo feedback para atingir um objetivo.
Sousa et al. (2023)	Plataformas sociotécnicas modernas que integram fluxos de dados, algoritmos inteligentes e colaboração humana para gerar valor estratégico e operacional.

Fonte: Elaborado pelo autor (2026)

A partir destas definições, estabelece-se para este trabalho que um Sistema de Informação é um ecossistema sociotécnico estruturado, que coordena a interação entre indivíduos, processos de negócio e tecnologias para capturar, transformar e disseminar dados de forma a gerar conhecimento acionável e valor estratégico para a organização.

É fundamental diferenciar os SI de conceitos correlatos para evitar sobreposições. A Tecnologia da Informação (TI) refere-se à infraestrutura física e lógica (*hardware*, redes, bancos de dados) que suporta um SI. A Tecnologia da Informação é o meio, enquanto o SI é a aplicação com propósito organizacional. A Ciência da Computação é o estudo teórico e matemático dos algoritmos e da computação em si. Já a Engenharia de *Software* foca nos métodos e processos para construir o software com qualidade (Sommerville, 2019). Operacionalmente, um SI em uma organização atua através de um ciclo contínuo, composto pelos seguintes procedimentos: Entrada (coleta de dados brutos do ambiente interno ou externo), Processamento (conversão desses dados em uma forma significativa através de cálculos ou classificações), Saída (transferência da informação processada para as pessoas ou atividades que a utilizarão) e *Feedback* (saída retornada para ajudar a refinar ou corrigir a fase de entrada) (Laudon; Laudon,

2014). Esse mecanismo auxilia a gestão ao fornecer visibilidade sobre as operações, identificar tendências, automatizar rotinas operacionais e embasar decisões táticas e estratégicas.

Recentemente, a Inteligência Artificial (IA) tem transformado o papel dos SI. A IA atua como um catalisador no processamento, permitindo que os sistemas não apenas reportem o passado, mas prevejam o futuro e tomem decisões autônomas (Sousa et al., 2023). Modelos preditivos, processamento de linguagem natural e algoritmos de aprendizado de máquina estão convertendo SI reativos em agentes proativos de conhecimento.

Para atender a diferentes níveis organizacionais, os SI são classificados em diversos tipos: Sistemas de Processamento de Transações (SPT) operam no nível diário e rotineiro; Sistemas de Informações Gerenciais (SIG) e Sistemas de Apoio à Decisão (SAD) fornecem relatórios e modelagens para gerentes médios; enquanto Sistemas de Apoio ao Executivo (SAE) orientam o nível estratégico (O'Brien; Marakas, 2013).

Dentre essas categorias, destacam-se os Sistemas de Gestão do Conhecimento (KMS - *Knowledge Management Systems*). Diferente de gerir dados estruturados (como em finanças), um KMS tem a finalidade específica de coletar, preservar e compartilhar o conhecimento e a expertise acumulados pelos colaboradores, conectando-se diretamente às necessidades de engenharia de *software* apresentadas neste estudo.

2.2 GESTÃO DO CONHECIMENTO

A Gestão do Conhecimento tem como objetivo organizacional identificar, criar, capturar, estruturar, compartilhar e preservar o conhecimento coletivo da empresa, transformando-o em um ativo estratégico que gera vantagem competitiva (Davenport; Prusak, 1998). O conhecimento é considerado um ativo estratégico pois, diferente de recursos físicos, ele não se deprecia com o uso. Pelo contrário, ele se expande e inova processos, sendo a principal fonte de diferencial em economias baseadas em serviços e tecnologia (Davenport; Prusak, 1998).

Para compreender a GC, é preciso distinguir informação de conhecimento. Segundo Davenport e Prusak (1998), os dados são registros simples e objetivos de fatos; a informação surge quando esses dados recebem contexto, categorização e

propósito. Já o conhecimento é fluido e complexo: é uma mistura de experiências, valores, informações contextuais e intuição especialista que fornece uma estrutura para avaliar e incorporar novas experiências e informações. Na engenharia de software, uma linha de código é dado; o log de erro é informação; mas o entendimento de por que aquela arquitetura específica foi escolhida é conhecimento.

Nonaka e Takeuchi (1995) estabeleceram a distinção fundamental entre o *conhecimento tácito* (pessoal, mental, difícil de formalizar, adquirido pela prática) e o *conhecimento explícito* (codificado, registrado em manuais, bancos de dados ou código-fonte). Para descrever a interação e a criação de conhecimento nas organizações, os autores propuseram o Modelo SECI, caracterizado por quatro etapas que formam uma espiral contínua:

- Socialização (Tácito para Tácito): Compartilhamento de experiências diretas. Em equipes de *software*, ocorre fortemente através do *Pair Programming* (programação em pares) ou discussões informais de arquitetura, onde desenvolvedores juniores absorvem a intuição dos seniores.
- Externalização (Tácito para Explícito): Articulação do conhecimento mental em conceitos tangíveis. É o grande desafio tecnológico, ocorrendo quando um desenvolvedor documenta uma API, escreve um relatório de *post-mortem* de um incidente, ou traduz sua lógica em código limpo.
- Combinação (Explícito para Explícito): Sistematização de conceitos em sistemas de conhecimento. Em TI, exemplifica-se pela consolidação de várias documentações de microsserviços em um único portal de desenvolvedor interno (IDP) estruturado.
- Internalização (Explícito para Tácito): Absorção do conhecimento formalizado em novas experiências e modelos mentais. Ocorre quando um novo funcionário estuda a documentação do sistema e a arquitetura e, a partir disso, torna-se capaz de desenvolver novas funcionalidades intuitivamente.

A relevância desse campo para equipes de *software* é direta, especialmente no que tange à retenção do conhecimento e à memória organizacional. Conforme Candal et al. (2022), as práticas de GC facilitam as atividades das equipes e influenciam positivamente o desenvolvimento. O oposto, a perda de conhecimento organizacional, muitas vezes gerada pela amnésia corporativa , representa um dos maiores riscos associados à alta rotatividade. Quando desenvolvedores saem sem externalizar seu conhecimento tácito, a memória organizacional sofre apagões que

resultam em retrabalho e perda de produtividade.

Para combater esse cenário, sistemas tecnológicos como plataformas de wikis, repositórios colaborativos de código, ferramentas de *issue tracking* e sistemas suportados por IA desempenham um papel vital. Eles atuam como repositórios dinâmicos da memória organizacional e automatizam parte do processo de externalização e combinação, transformando códigos isolados em mapas arquiteturais pesquisáveis.

2.3 ENGENHARIA DE SOFTWARE

A Engenharia de Software é a disciplina que estabelece princípios, métodos e ferramentas para o desenvolvimento sistemático e disciplinado de software, abrangendo desde a especificação inicial até a manutenção do sistema em uso (Sommerville, 2019). Seu processo se organiza em um ciclo de vida composto pelas fases de especificação, projeto, implementação, validação e evolução. Cada uma produz artefatos que precisam ser compreensíveis por todos os membros da equipe. Essa perspectiva é reforçada por Martin (2009), para quem o código é lido muito mais vezes do que escrito, sendo sua legibilidade uma necessidade funcional.

Sommerville (2019) destaca que a manutenção frequentemente consome mais recursos do que o desenvolvimento original, tornando a compreensibilidade um fator crítico. As dificuldades amplificam-se com o crescimento dos sistemas. Martin (2009) identifica que o acoplamento excessivo e funções com múltiplas responsabilidades tornam um sistema resistente à transferência de conhecimento.

Para endereçar essas falhas de compreensibilidade sem depender inteiramente da documentação manual exaustiva que rapidamente fica defasada , a Engenharia de Software recorre à análise estática de código e à recuperação de conhecimento. A análise estática inspeciona o código-fonte sem executá-lo, sendo o primeiro passo desse processo a conversão do texto bruto em uma Árvore de Sintaxe Abstrata, ou AST (Abstract Syntax Tree). AST é uma representação hierárquica da estrutura sintática do código, na qual detalhes superficiais da escrita humana como pontuação, chaves e espaços em branco são omitidos. Cada nó desta árvore denota um construto fundamental da linguagem, como declarações de variáveis, operadores lógicos ou chamadas de função.

É através da travessia dos nós da AST que ferramentas modernas

conseguem processar arquivos em alta velocidade e concorrência, compreendendo a semântica do programa de forma programática. Ao analisar como um nó de "chamada de função" se conecta a um nó de "declaração de módulo", o sistema identifica automaticamente as relações de dependência, os padrões arquiteturais e as métricas de complexidade.

Essa abordagem tecnológica permite a Recuperação de Conhecimento (*Knowledge Recovery*). Ao invés de o desenvolvedor descrever o que o sistema faz, a ferramenta de análise estática extrai o que o sistema realmente faz, gerando grafos de dependências modulares e, quando aliadas a modelos de Inteligência Artificial, sumarizações automáticas das regras de negócio. Essa mecanização desloca o esforço da documentação reativa para a geração proativa de contexto, constituindo o pilar técnico da solução proposta por este trabalho para mitigar a perda de memória organizacional nas equipes de desenvolvimento.

3. PROCEDIMENTOS METODOLÓGICOS

Esta pesquisa classifica-se, quanto à sua natureza, como uma pesquisa aplicada, pois visa gerar conhecimentos para aplicação prática dirigidos à solução de um problema específico: a dificuldade de retenção e gestão do conhecimento em equipes de desenvolvimento de software. Quanto à abordagem, adota um formato qualiquantitativo, unindo a avaliação subjetiva da qualidade do conhecimento gerado (qualitativo) com métricas de desempenho e estruturação algorítmica (quantitativo).

O procedimento metodológico está estruturado em três etapas principais de desenvolvimento tecnológico, seguidas por uma etapa de validação:

1. Mapeamento e Leitura da AST (Extração Estrutural): A primeira etapa consiste na construção do motor de análise estática da ferramenta. O sistema será projetado para receber o código-fonte de repositórios e, por meio de *parsers* específicos da linguagem de programação alvo, converter os arquivos de texto bruto em Árvores de Sintaxe Abstrata (AST). A partir da travessia hierárquica desses nós, o algoritmo será responsável por identificar módulos, funções, classes e, criticamente, as relações de importação e chamadas de métodos entre eles, criando um inventário estrutural preciso do ecossistema do software.

2. Aplicação de Inteligência Artificial (Sumarização): De posse da estrutura lógica extraída pela AST, a segunda etapa envolverá a integração da ferramenta com modelos de linguagem de grande porte (LLMs - *Large Language Models*) via Interface de Programação de Aplicações (API). Em vez de alimentar a IA com o código inteiro (o que excede limites de contexto e gera ruído), a ferramenta enviará blocos lógicos estruturados (funções e módulos específicos isolados na etapa anterior) junto com instruções de contexto (*prompts*). O objetivo é que a IA retorne uma sumarização textual que traduza a lógica técnica em regras de negócio compreensíveis, gerando a "documentação viva".

3. Desenvolvimento da Interface Visual Interativa: Para tornar o conhecimento acessível aos desenvolvedores, a terceira etapa compreende a construção da interface de usuário (*front-end*). Os dados extraídos nas etapas anteriores serão renderizados de duas formas principais: (a) como um catálogo interativo, permitindo a pesquisa textual sobre os resumos gerados pela IA; e (b) como grafos de dependência dinâmicos (utilizando representação de nós e arestas), onde os usuários poderão visualizar o impacto de alterações arquiteturais e como os módulos se interconectam de forma holística.

4. Validação da Ferramenta: A avaliação da eficácia da aplicação será conduzida por meio de testes em cenários reais simulados. Serão selecionados repositórios de código de código aberto (*open-source*) de diferentes níveis de complexidade para serem processados pela ferramenta. A validação ocorrerá com base em dois critérios centrais:

- *Acurácia Estrutural:* Verificação manual se o grafo de dependências gerado pela leitura da AST reflete corretamente a arquitetura real do software.
- *Qualidade da Sumarização:* Avaliação heurística dos textos gerados pela IA, observando se eles explicam de maneira clara e correta o propósito das funções mapeadas, comprovando assim sua utilidade na redução da curva de aprendizado de um novo desenvolvedor frente a um sistema desconhecido.

Referências

CANDAL, Diego Marconi et al. Influência de Práticas de Gestão do Conhecimento Aplicadas ao Desenvolvimento Ágil de Software. **RISTI — Revista Ibérica de Sistemas e Tecnologias de Informação**, n. 48, p. 74–88, dez. 2022.

DAVENPORT, T. H.; PRUSAK, L. **Conhecimento Empresarial: Como as Organizações Gerenciam o Seu Capital Intelectual**. Rio de Janeiro: Campus, 1998.

LAUDON, K. C.; LAUDON, J. P. **Sistemas de Informações Gerenciais**. 11. ed. São Paulo: Pearson, 2014.

MARTIN, R. C. **Código Limpo: Habilidades Práticas do Agile Software**. Rio de Janeiro: Alta Books, 2009.

NONAKA, I.; TAKEUCHI, H. **Criação de Conhecimento na Empresa**. Rio de Janeiro: Elsevier, 1995.

O'BRIEN, J. A.; MARAKAS, G. M. **Sistemas de Informação e as Decisões Gerenciais na Era da Internet**. 4. ed. São Paulo: Saraiva, 2013.

PRESSMAN, R. S.; MAXIM, B. R. **Engenharia de Software: Uma Abordagem Profissional**. 9. ed. Porto Alegre: AMGH, 2021.

SOMMERVILLE, I. **Engenharia de Software**. 10. ed. São Paulo: Pearson, 2019.

SOUSA, R. et al. O Papel da Inteligência Artificial nos Sistemas de Informação Modernos. **Revista Brasileira de Computação Aplicada**, v. 15, n. 2, p. 45-60, 2023.

STAIR, R. M.; REYNOLDS, G. W. **Princípios de Sistemas de Informação**. 13. ed. São Paulo: Cengage Learning, 2020.